

이끼잡 — 취업 준비 자동화 풀스택 플랫폼

1. 프로젝트 소개

이끼잡은 채용 공고 탐색부터 AI 문서 생성, 이력서 연동, 자동 지원까지 취업 준비 과정을 하나의 흐름으로 연결한 풀스택 취업 지원 플랫폼입니다.

핵심 기능

- 4개 채용 사이트(사람인·잡코리아·잡플래닛·링크리어) 공고 자동 수집 및 통합 조회
- AI 기반 자기소개서·포트폴리오 생성 (비동기 큐 + 실시간 알림)
- 외부 사이트 이력서 가져오기 및 역동기화
- Playwright 기반 원클릭 자동 지원 + 사후 검증
- 관리자 대시보드 (크롤링 제어, 유저 관리, 통계)

담당 역할

- 백엔드 아키텍처 설계 및 전체 구현
- 채용 사이트 크롤러 및 자동 지원 로직 설계
- AI 문서 생성 비동기 파이프라인 구축
- 성능 최적화 및 운영 안정화
- 보안 정책 개선 및 인증 구조 고도화

기술 스택

레이어	기술
Backend	Java 21, Spring Boot 3.4.3, Spring Security, JPA
Frontend	Next.js 16.1.6, React 19.2.3, TypeScript, Tailwind CSS 4
Database	PostgreSQL 17, Redis 7
Crawling	Playwright 1.49.0, Jsoup 1.17.2
Security	JWT 0.12.6, AES-256-GCM
Infra	AWS EC2, Nginx (TLS/HSTS), PM2
실시간	Spring WebSocket, STOMP.js 7.3.0, SockJS 1.6.1

2. 기술 구조와 설계 판단

2-1. 사이트별 차이를 흡수하는 Strategy Pattern 설계

배경 및 문제정의

- 상황: 4개 채용 사이트는 DOM 구조, 지원 방식, 인증 흐름이 모두 다름
- 문제: if-else 분기로 처리하면 사이트 추가/변경마다 기존 코드 전체를 흔들게 됨

솔루션: 공통 인터페이스 + 사이트별 구현체 분리

서비스 계층은 오케스트레이션만 담당하고, 사이트별 세부 동작은 Strategy 구현체로 격리했습니다. 신규 사이트 추가 시 기존 코드 수정 없이 구현체만 추가하면 되는 구조입니다.

```
// SiteParser.java — 크롤러 전략 인터페이스
public interface SiteParser {
    String getSiteName();
    String buildSearchUrl(String keyword, String jobCategory);
    Locator getListItems(Page page);
    CrawledJobData parseJobData(Page listPage, Locator item, String requestedJobCategory);
}
```

```

        boolean goToNextPage(Page page, int currentPageNum);
    }

```

```

// SaraminParser.java - 사람인 구현체
@Component
public class SaraminParser implements SiteParser {
    @Override
    public String getSiteName() { return "SARAMIN"; }

    @Override
    public String buildSearchUrl(String keyword, String jobCategory) {
        // 사람인 전용 URL 구성
    }
    // ... 사이트별 DOM 파싱 로직
}

```

같은 패턴을 크롤러(SiteParser), 자동 지원(AutoApplyProvider), 이력서 동기화(ResumeSyncProvider), AI 프로필 빌더 (AiProfileBuilder)에 일관 적용했습니다.

2-2. Redis 태스크 큐 기반 비동기 파이프라인

배경 및 문제정의

- 상황: AI 자소서/포트폴리오 생성은 10~30초 소요
- 문제: 동기 REST로 처리하면 화면이 멈추고 사용자 경험이 크게 저하됨

기술 선정: 동기 처리 vs 비동기 큐

항목	동기 REST	Redis 태스크 큐
사용자 경험	요청 중 화면 멈춤	즉시 taskId 반환, 백그라운드 처리
장시간 작업	타임아웃 위험	큐에서 독립 처리
완료 알림	불가 (폴링만 가능)	WebSocket 즉시 + 폴링 복구
확장성	서버 스레드에 종속	큐 기반 워커 분리 가능

구현

```

// AiTaskQueue.java - Redis 기반 태스크 큐
public String enqueue(String taskType, Long userId) {
    String taskId = UUID.randomUUID().toString().substring(0, 8);
    String key = TASK_PREFIX + taskId;

    Map<String, String> task = new ConcurrentHashMap<>();
    task.put("status", PENDING);
    task.put("type", taskType);
    task.put("userId", String.valueOf(userId));

    redisTemplate.opsForHash().putAll(key, task);
    redisTemplate.expire(key, TASK_TTL); // 10분 TTL
    return taskId;
}

public void complete(String taskId, Long userId, String result) {
    updateRedis(taskId, COMPLETED, result);
}

```

```
    notifyUser(userId, taskId, COMPLETED, result); // WebSocket 알림
}
```

```
// WebSocket STOMP 완료 알림
private void notifyUser(Long userId, String taskId, String status, String result) {
    Map<String, Object> message = Map.of(
        "taskId", taskId, "status", status,
        "result", result != null ? result : "");
    messagingTemplate.convertAndSend("/topic/ai/" + userId, message);
}
```

네트워크 불안정 시를 대비해 프론트엔드에서 3초 간격 폴링 복구 경로도 함께 구성했습니다.

2-3. 공고 원문 HTML 보존 + 이중 소독

배경 및 문제정의

- 상황: 채용 공고의 표·줄바꿈·강조 구문은 그 자체가 의미를 가짐
- 문제: `innerText()` 로 수집하면 table이 탭 문자로 찢어지고 구조가 무너짐

기술 선정: 텍스트 추출 vs HTML 보존

항목	<code>innerText()</code> 추출	innerHTML 원본 보존
표 구조	탭/줄바꿈으로 무너짐	완전 보존
검색 품질	자격요건/우대사항 혼합	구조 유지로 정확도 높음
보안	XSS 위험 낮음	이중 소독 필요
저장 용량	~5KB	~10KB

솔루션

`innerHTML` 원본 저장 후 서버(`Jsoup Safelist.relaxed()`)와 클라이언트(`DOMPurify 3.3.3`)로 이중 소독을 적용했습니다. 표 구조 보존율은 100%를 달성했으며, 저장 용량 증가는 TEXT 컬럼 기준 운영상 수용 가능한 수준이었습니다.

2-4. 선별적 Redis 캐시 + Cache Stampede 방지

정렬·필터 조합이 많은 목록 API는 캐시 키가 과도하게 증가하므로, 반복 접근이 많고 응답이 안정적인 공고 통계·상세 조회에만 5분 TTL 캐시를 선별 적용했습니다. PostgreSQL을 원본 저장소로, Redis를 읽기 최적화 계층으로 분리하는 구조를 택했습니다.

2-5. AES-256-GCM 양방향 암호화

배경 및 문제정의

- 상황: 자동 지원 시 외부 사이트 계정 정보를 다시 꺼내 써야 함
- 문제: 단방향 해시로는 기능 불가, Base64 난독화는 보안 대책이 아님

구현

```
// AesEncryptor.java - AES-256-GCM 암호화
public String encrypt(String plainText) {
    byte[] iv = new byte[IV_LENGTH];
    new SecureRandom().nextBytes(iv);

    Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
    cipher.init(Cipher.ENCRYPT_MODE, keySpec, new GCMParameterSpec(GCM_TAG_LENGTH, iv));
```

```

byte[] encrypted = cipher.doFinal(plainText.getBytes());

// IV + 암호문을 합쳐서 Base64 인코딩
ByteBuffer buffer = ByteBuffer.allocate(IV_LENGTH + encrypted.length);
buffer.put(iv);
buffer.put(encrypted);
return Base64.getEncoder().encodeToString(buffer.array());
}

```

매 암호화마다 새 IV를 생성하여 동일 평문이라도 다른 암호문이 출력됩니다. Spring Security + JWT 0.12.6 인증 흐름 위에 적용했습니다.

3. 주요 문제 해결 사례

3-1. 병렬 수집 실패율 31~63% → 성공률 100%

배경 및 문제정의

- **상황:** 상세 공고 병렬 수집을 위해 3개 워커가 하나의 브라우저 연결을 공유
- **문제:** 병렬 요청이 겹치면 WebSocket 충돌으로 실패율 31~63%까지 상승. 재시도 로직으로도 해결 불가

분석

처음에는 네트워크 불안정으로 판단했지만, 로그를 쌓아 확인한 결과 원인은 **실행 단위 경계가 모호한 구조적 문제**였습니다. 하나의 브라우저 WebSocket을 여러 워커가 동시에 사용하고 있었습니다.

솔루션: 워커별 독립 브라우저 인스턴스 분리

```

// PlaywrightManager.java - 독립 워커 생성
public class PlaywrightWorker implements AutoCloseable {
    private final Playwright workerPlaywright;
    private final Browser workerBrowser;

    PlaywrightWorker() {
        this.workerPlaywright = Playwright.create(); // 독립 Playwright
        this.workerBrowser = workerPlaywright.chromium().launch(// 독립 Chromium 프로세스
            new BrowserType.LaunchOptions()
                .setHeadless(false)
                .setArgs(List.of(
                    "--disable-blink-features=AutomationControlled",
                    "--no-sandbox", "--disable-dev-shm-usage"))));
    }
}

```

```

// CrawlerEngine.java - 워커별 독립 실행
futures.add(pool.submit(() -> {
    try (PlaywrightManager.PlaywrightWorker worker = playwrightManager.createIsolatedWorker()) {
        try (BrowserContext ctx = worker.createStealthContext()) {
            // 독립 브라우저·독립 WebSocket·독립 프로세스에서 상세 수집
        }
    }
}));

```

병렬도는 `crawler.detail.concurrency=3` 설정값으로 제어했습니다.

결과

지표	Before	After
상세 수집 성공률	~40%	100%
50건 처리 시간	-	~90초 유지
브라우저 메모리	~200MB	~500MB

메모리 증가는 있었지만, 설정값으로 워커 수를 제어하여 서버 자원 내에서 안정적으로 운영할 수 있었습니다.

3-2. AI 생성 타임아웃 460초 → 20초

배경 및 문제정의

- 상황: 30명 동시 AI 문서 생성 요청
- 문제: Semaphore(5) 앞에 대기 스레드 최대 120개 적체 → 30초 타임아웃으로 대부분 실패

분석

단순히 Semaphore 값을 30으로 올리면 AI 서버 처리량을 초과하여 전체 응답이 더 나빠질 위험이 있었습니다. 병목은 코드가 아니라 인프라 처리량과 맞지 않는 동시성 설정값에 있었습니다.

솔루션: 인프라 기반 동시성 설정 외부화

```
// OpenClawAiGenerator.java - Semaphore 외부 설정 기반 초기화
@Value("${ai.max-concurrent-requests:15}")
private int maxConcurrentRequests;

private Semaphore aiSemaphore;

@PostConstruct
private void initSemaphore() {
    aiSemaphore = new Semaphore(maxConcurrentRequests);
    log.info("[AI] Semaphore 초기화: 동시 요청 최대 {}개", maxConcurrentRequests);
}
```

```
// 사용 시 - 30초 타임아웃 + 자동 해제
private String callHttpApi(String prompt, List<String> imageUrls) throws Exception {
    if (!aiSemaphore.tryAcquire(30, TimeUnit.SECONDS)) {
        throw new RuntimeException("AI 요청 대기 시간 초과");
    }
    try {
        return callHttpApiInternal(prompt, imageUrls);
    } finally {
        aiSemaphore.release();
    }
}
```

AI 인스턴스 3개 × 인스턴스당 5개 처리량 = ai.max-concurrent-requests=15

결과

지표	Before	After
타임아웃 대기	~460초	~20초
동시성 제어	코드 하드코딩	설정값 외부화 (재배포 불필요)

3-3. Cache Stampede → Redis 분산 락

배경 및 문제정의

- 상황: 캐시 만료 순간에 동시 요청이 한꺼번에 DB로 유입
- 문제: 5개 동시 요청만으로도 동일 데이터 쿼리 20회 중복 실행 → 연결 풀 고갈

솔루션: `setIfAbsent` 분산 락

```
// Cache Stampede 방지: 1개만 DB 조회, 나머지는 캐시 재확인
Boolean locked = redisTemplate.opsForValue()
    .setIfAbsent(STATS_LOCK_KEY, "1", LOCK_TTL); // 10초 TTL 분산 락

if (Boolean.FALSE.equals(locked)) {
    // 락 획득 실패 → 500ms 대기 후 캐시 재확인
    Thread.sleep(500);
    Object retry = redisTemplate.opsForValue().get(STATS_CACHE_KEY);
    if (retry instanceof String json) {
        return objectMapper.readValue(json, new TypeReference<>() {});
    }
}
// 락 획득 성공 → DB 조회 후 캐시 저장
```

결과

지표	Before	After
5개 동시 요청 시 DB 쿼리	20회	4회
캐시 저장	5회 (중복)	1회
동시 접속 한계	~100명	300~500명
DB 부하	기준	80% 감소

3-4. 검색 병목 → 인덱스 전략

배경 및 문제정의

- 상황: 공고 검색은 ILIKE 부분 문자열 검색 + 정렬이 결합된 쿼리
- 문제: 데이터 증가에 따라 Full Table Scan 발생 가능성

솔루션

- `title`, `company` → `pg_trgm GIN` 인덱스 (부분 문자열 최적화)
- `source` + `created_at DESC` → `복합 인덱스` (정렬 경로 최적화)
- HikariCP `maximum-pool-size` 10 → 30 (연결 대기 병목 완화)

결과 (k6 부하테스트, 200명 동시접속)

지표	결과
성공률	100%
에러율	0%
p95 응답	5.79ms
키워드 검색	0.37ms
OFFSET 1000 페이지	0.95ms

3-5. 자동 지원 허위 성공 → 사후 검증

외부 사이트 자동 지원은 입력 완료만으로 성공을 보장할 수 없었습니다. jQuery 이벤트 위임, hidden 필드, 동적 DOM 환경에서 14번 중 13번 실패하면서도 성공으로 판정되는 사례가 발생했습니다.

이를 해결하기 위해 사이트별 지원 전략에 **결과 검증 단계와 사후 검증 스케줄러**를 추가하여, 제출 이후에도 실제 성공 여부를 재확인하는 구조로 전환했습니다.

3-6. 보안 등급 C+ → B

항목	Before	After
외부 계정 저장	Base64	AES-256-GCM
관리자 계정	코드 하드코딩	DB + 승인 흐름
토큰 갱신	30분마다 재로그인	JWT 자동 갱신 (만료 2분 전)
보안 헤더	없음	HSTS, X-Frame-Options, X-Content-Type-Options
민감정보	응답에 노출	마스킹 적용
보안 등급	C+ (5.5/10)	B (7.5/10)

4. 회고

이 프로젝트는 기능 개발에서 끝나지 않고, 외부 의존성이 큰 서비스를 **실제 운영 가능한 수준까지 끌어올린 경험**이었습니다.

PostgreSQL, Redis, WebSocket, Playwright, Spring Security처럼 성격이 다른 기술을 한 흐름으로 묶어 병목과 장애를 수치로 줄였으며, **왜 이 선택이 맞는지 설명할 수 있는 방식**으로 개발했습니다.